Український католицький університет

Statistics and Econometrics

Завдання 2.

Інференційна статистика

Аналітичний звіт

Виконав: **Владислав Григола**

2026

Зміст

1 Закон великих чисел, довірчі інтервали та ЦГТ **3**

2 Оцінювання максимальною правдоподібністю **10**

3 Перевірка гіпотез про продуктові метрики **13**

4 Емпіричний рівень помилки першого роду **18**

5 Потужність критерію Лілієфорса **20**

Дані та домовленості. У звіті виконано всі п’ять частин наданого завдання. У самому файлі умова має заголовок «Задача 4»; цей звіт оформлено як навчальне Завдання 2. Для частин 2 і 3 використано наданий `app_metrics.csv`: 250 спостережень без пропусків. Дані не змінювалися. Рівень значущості дорівнює $\alpha = 0.05$; для частини 5 це явно прийнята домовленість. В основних симуляціях використано початкове значення генератора 42; для додаткової контрольної перевірки — 20261014. Вибіркові дисперсії обчислено з дільником $n - 1$.

Час у частині 1 вимірюється в умовних одиницях, оскільки одиницю в умові не вказано; Session_Time у частині 3 вимірюється у хвилинах. Валюту Revenue та Purchase_Amount не задано. Крапка використовується як десятковий роздільник. Наведені числові результати округлено; розрахунки виконано з повною машинною точністю. Кожен підпункт містить окремо умову, код Python та результати з інтерпретацією. Умови подано зі збереженням вимог і нумерації оригіналу. Код наведено з виконаних ноутбуків; кольором виділено ключові слова, рядки, числа та коментарі. Повні ноутбуки зі збереженими результатами підготовлено для індивідуального захисту.

Спільне налаштування Python

Цей блок виконується на початку кожного ноутбука в окремому чистому ядрі. Він імпортує бібліотеки, знаходить каталог завдання, створює каталоги результатів та ініціалізує генератор випадкових чисел. Наступні блоки всередині кожної частини виконуються послідовно. Команда `display` доступна в середовищі Jupyter.

Код Python

```python
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

TASK_DIR = Path.cwd()
if (TASK_DIR / "labs" / "task2").is_dir():
    TASK_DIR = TASK_DIR / "labs" / "task2"
elif TASK_DIR.name == "notebooks":
    TASK_DIR = TASK_DIR.parent
INPUT_DIR = TASK_DIR / "input"
PDF_DIR = TASK_DIR / "pdf"
FIGURES_DIR = PDF_DIR / "figures"
SEED = 42
ALPHA = 0.05
rng = np.random.default_rng(SEED)
sns.set_theme(style="whitegrid")

FIGURES_DIR.mkdir(parents=True, exist_ok=True)
RESULTS_DIR = PDF_DIR / "results"
RESULTS_DIR.mkdir(exist_ok=True)
plt.rcParams.update({"figure.dpi": 120, "savefig.bbox": "tight"})
```

Частина 1. Закон великих чисел, довірчі інтервали та ЦГТ

Підготовка частини 1

Задаємо параметри Gamma-розподілу, обсяги вибірок та кількість повторень.

Код Python

```python
SHAPE = 2
SCALE = 5
TRUE_MEAN = 10
TRUE_VARIANCE = 50
SAMPLE_SIZES = 5 ** np.arange(1, 9)  # 5¹ through 5⁸, not 51 through 58
REPLICATIONS = 1000
SMALL_N = 4
LARGE_N = 100
```

1(a). Вибіркові моменти та закон великих чисел

Умова завдання

Нехай час між сесіями має Gamma-розподіл із параметрами форми $k = 2$ і масштабу $\theta = 5$, тому $\mu = 10$, $\sigma^2 = 50$. Змоделюйте вибірки розміром $n = 5^1, 5^2, \ldots, 5^8$. Для кожної обчисліть вибіркове середнє та вибіркову дисперсію. Побудуйте графік вибіркових середніх як функцію від n з логарифмічною шкалою осі X. Поясніть результат за допомогою слабкого закону великих чисел.

Код Python

```python
samples = [rng.gamma(shape=SHAPE, scale=SCALE, size=int(n)) for n in SAMPLE_SIZES]
moments = pd.DataFrame({
    "n": SAMPLE_SIZES,
    "mean": [x.mean() for x in samples],
    "variance": [x.var(ddof=1) for x in samples],
})
display(moments.round(6))
moments.to_csv(RESULTS_DIR / "part1_moments.csv", index=False)

fig, ax = plt.subplots(figsize=(8, 4.2))
ax.plot(moments["n"], moments["mean"], "o-", label="Вибіркове середнє")
ax.axhline(TRUE_MEAN, color="black", linestyle="--", label=r"$\mu=10$")
ax.set(xscale="log", xlabel="Обсяг вибірки n (логарифмічна шкала)",
       ylabel="Середній час (умовні одиниці)", title="Закон великих чисел: середні Gamma(2, 5)")
ax.legend()
fig.tight_layout()
fig.savefig(FIGURES_DIR / "part1_lln.pdf")
plt.show()
```

Результати та інтерпретація

Використано незалежні, а не вкладені вибірки: для кожного $n = 5^1, \ldots, 5^8$ генерується нова вибірка. Параметр $\theta = 5$ є масштабом. Тому

$$\mu = k\theta = 10, \qquad \sigma^2 = k\theta^2 = 50.$$

Обчислено

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i, \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

При $n = 5$ отримано $\bar{x} = 11.4887$, а при $n = 390625$ маємо $\bar{x} = 9.9963$ та $s^2 = 49.7777$. Великі вибірки дають середні, близькі до 10. За слабким законом великих чисел для кожного $\varepsilon > 0$ маємо $P(|\bar{X}_n - 10| > \varepsilon) \to 0$. Графік узгоджується з цим твердженням, але не доводить його. Збіжність не означає монотонного зменшення похибки в кожній реалізації. Для оцінки середнього часу між сесіями більша вибірка зазвичай дає точніший результат.

Табл. 1: Вибіркові моменти незалежних вибірок із розподілу Gamma(2, 5).

n	$\bar{x}$	s^2
5	11.488704	9.876565
25	8.890425	21.964613
125	10.097676	63.121252
625	9.831200	45.690043
3125	10.120737	49.760696
15625	10.057645	51.695993
78125	9.996694	50.433119
390625	9.996343	49.777707

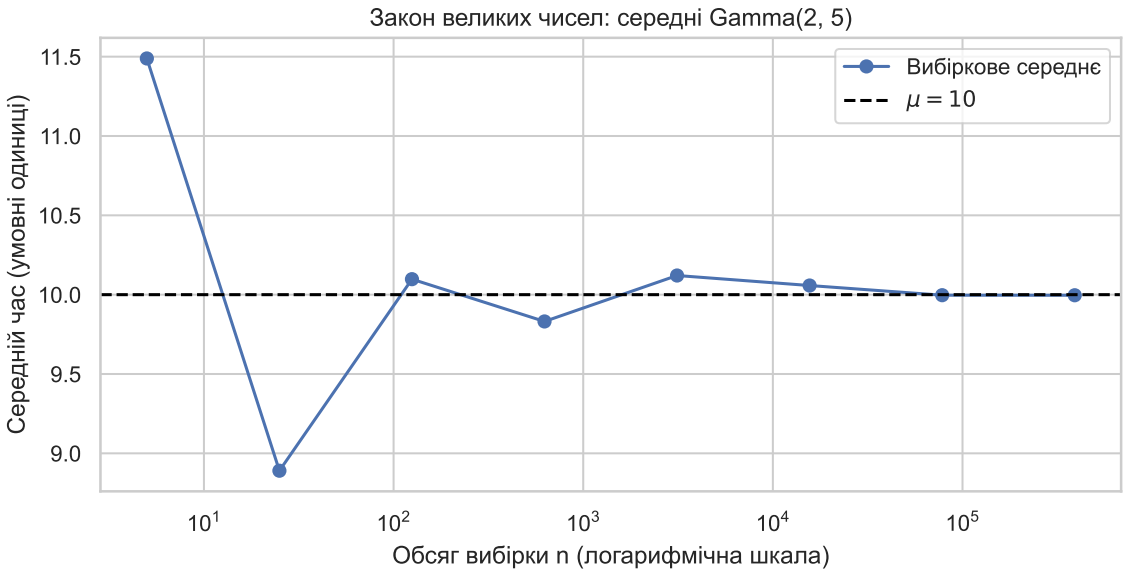


Рис. 1: Траєкторія вибіркових середніх; вісь обсягу вибірки логарифмічна.

1(б). Довірчі інтервали для середнього

Умова завдання

Для кожної вибірки вручну обчисліть 95% довірчий інтервал для середнього μ і додайте його на графік. Зробіть це двічі: з відомим теоретичним стандартним відхиленням σ та з оціненим вибірковим стандартним відхиленням s. Обговоріть спостереження та теоретично поясніть результати при малих і великих n.

Код Python

```
z_critical = stats.norm.ppf(1 - ALPHA / 2)
t_critical = stats.t.ppf(1 - ALPHA / 2, df=SAMPLE_SIZES - 1)
known_margin = z_critical * np.sqrt(TRUE_VARIANCE / SAMPLE_SIZES)
estimated_margin = t_critical * np.sqrt(moments["variance"] / SAMPLE_SIZES)
intervals = moments[["n", "mean"]].copy()
```

Код Python (продовження)

```python
intervals["known_lower"] = moments["mean"] - known_margin
intervals["known_upper"] = moments["mean"] + known_margin
intervals["estimated_lower"] = moments["mean"] - estimated_margin
intervals["estimated_upper"] = moments["mean"] + estimated_margin
display(intervals.round(6))
intervals.to_csv(RESULTS_DIR / "part1_intervals.csv", index=False)

fig, ax = plt.subplots(figsize=(8, 4.8))
ax.errorbar(SAMPLE_SIZES, moments["mean"], yerr=estimated_margin,
            fmt="o-", capsize=6, alpha=0.75, label="Оцінене s: t-інтервал")
ax.errorbar(SAMPLE_SIZES, moments["mean"], yerr=known_margin,
            fmt="s", capsize=3, color="darkorange", label="Відоме σ: z-інтервал")
ax.axhline(TRUE_MEAN, color="black", linestyle="--", label=r"$\mu=10$")
ax.set(xscale="log", xlabel="Обсяг вибірки n (логарифмічна шкала)",
       ylabel="Середній час (умовні одиниці)", title="Наближені 95% довірчі інтервали")
ax.legend()
fig.tight_layout()
fig.savefig(FIGURES_DIR / "part1_intervals.pdf")
plt.show()
```

Результати та інтерпретація

Інтервали побудовано за формулами

$$I_\sigma = \left[\bar{x} - z_{0.975}\frac{\sqrt{50}}{\sqrt{n}},\ \bar{x} + z_{0.975}\frac{\sqrt{50}}{\sqrt{n}}\right], \qquad z_{0.975} \approx 1.9600,$$

$$I_s = \left[\bar{x} - t_{0.975,n-1}\frac{s}{\sqrt{n}},\ \bar{x} + t_{0.975,n-1}\frac{s}{\sqrt{n}}\right].$$

Другий інтервал використовує оцінене стандартне відхилення та квантиль Стьюдента. Обидва інтервали тут **наближені**: Gamma-дані не є нормальними, а точний t-інтервал вимагає нормальності генеральної сукупності. При малих n фактичне покриття може відрізнятися від 95%.

Для $n = 5$ отримано $I_\sigma = [5.2908;\ 17.6867]$ та $I_s = [7.5865;\ 15.3909]$. У цій вибірці $s^2 = 9.8766$ значно менша за 50, тому інтервал з оціненим s вужчий, попри більший t-квантиль. Він не зобов'язаний завжди бути ширшим. При великих n маємо $s \to \sigma$, t-квантиль наближається до z-квантиля, а ЦГТ покращує нормальне наближення. Ширина інтервалів зменшується приблизно як $1/\sqrt{n}$.

У цій реалізації всі інтервали містять 10. Вісім інтервалів не є оцінкою довгострокового 95% покриття. Рівень 95% характеризує частку інтервалів, які накривали б фіксоване μ при багаторазовому повторенні процедури за відповідних умов.

Табл. 2: Наближені 95% довірчі інтервали.

n	I_σ	I_s
5	[5.2908; 17.6867]	[7.5865; 15.3909]
25	[6.1186; 11.6622]	[6.9559; 10.8250]
125	[8.8581; 11.3373]	[8.6912; 11.5042]
625	[9.2768; 10.3856]	[9.3002; 10.3622]
3125	[9.8728; 10.3687]	[9.8733; 10.3682]
15625	[9.9468; 10.1685]	[9.9449; 10.1704]
78125	[9.9471; 10.0463]	[9.9469; 10.0465]
390625	[9.9742; 10.0185]	[9.9742; 10.0185]

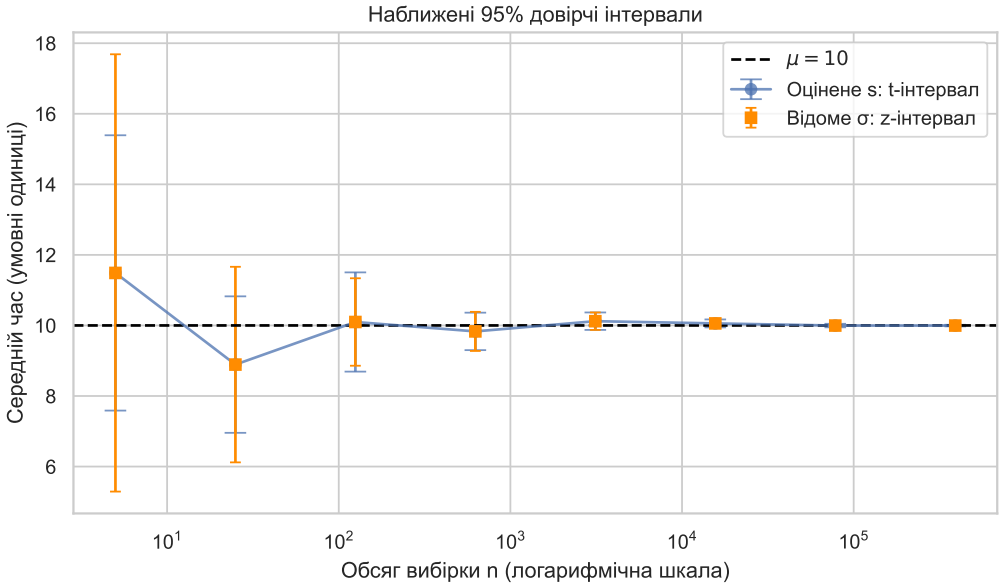


Рис. 2: Інтервали з відомим та оціненим стандартним відхиленням.

1(в). Розподіли середніх і дисперсій при обсязі 4

Умова завдання

Змоделюйте $b = 1000$ вибірок обсягу $n = 4$ з асиметричного розподілу $\Gamma(2, 5)$. Для кожної оцініть і збережіть середнє та дисперсію. Побудуйте гістограму 1000 вибіркових середніх і гістограму 1000 вибіркових дисперсій.

Код Python

```
small_samples = rng.gamma(shape=SHAPE, scale=SCALE, size=(REPLICATIONS, SMALL_N))
small_means = small_samples.mean(axis=1)
small_variances = small_samples.var(axis=1, ddof=1)
repeated_moments = pd.DataFrame({"mean": small_means, "variance": small_variances})
repeated_moments.to_csv(RESULTS_DIR / "part1_repeated_moments.csv", index=False)
display(repeated_moments.agg(["mean", "var"]).round(6))

fig, axes = plt.subplots(1, 2, figsize=(10, 4.2))
axes[0].hist(small_means, bins=30, edgecolor="white")
axes[0].set(xlabel="Вибіркове середнє (умовні одиниці)", ylabel="Кількість вибірок",
            title="1000 середніх, n = 4")
axes[1].hist(small_variances, bins=30, edgecolor="white", color="darkorange")
axes[1].set(xlabel="Вибіркова дисперсія (ум. од.²)", ylabel="Кількість вибірок",
            title="1000 дисперсій, n = 4")
fig.tight_layout()
fig.savefig(FIGURES_DIR / "part1_histograms.pdf")
plt.show()
```

Результати та інтерпретація

Згенеровано 1000 незалежних вибірок обсягу 4. Для кожної збережено $\bar{x}$ та незміщену дисперсію s^2 з дільником $n - 1 = 3$. Середнє 1000 оцінок $\bar{x}$ дорівнює 10.1350, а середнє 1000 оцінок s^2 становить 50.9810. Вони близькі до 10 і 50 відповідно. Обидві гістограми мають праву асиметрію; дисперсії особливо нестійкі для такої малої вибірки. Окремі великі значення збільшують правий хвіст гістограми дисперсій.

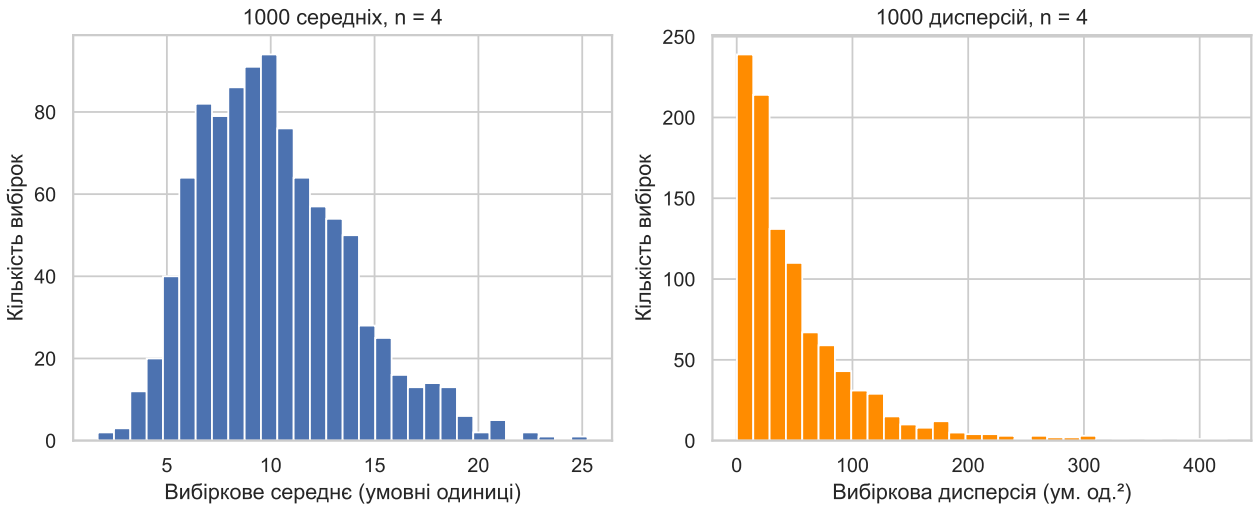


Рис. 3: Гістограми 1000 середніх та 1000 незміщених вибіркових дисперсій.

1(г). Аналітичні моменти вибіркового середнього

Умова завдання

Використовуючи властивості розподілу, обраного в пункті 1(в), аналітично обчисліть

$$E\left(\frac{X_1+X_2+X_3+X_4}{4}\right), \qquad \mathrm{Var}\left(\frac{X_1+X_2+X_3+X_4}{4}\right).$$

Код Python

```python
mean_of_mean = SHAPE * SCALE
variance_of_mean = SHAPE * SCALE**2 / SMALL_N
mean_comparison = pd.DataFrame({
    "quantity": ["E(mean)", "Var(mean)", "E(sample variance)"],
    "theoretical": [mean_of_mean, variance_of_mean, TRUE_VARIANCE],
    "simulation": [small_means.mean(), small_means.var(ddof=1), small_variances.mean()],
})
display(mean_comparison.round(6))
mean_comparison.to_csv(RESULTS_DIR / "part1_comparison.csv", index=False)
```

Результати та інтерпретація

Нехай $X_1, \ldots, X_4$ незалежні й однаково розподілені як $\Gamma(2, 5)$. За лінійністю сподівання

$$E(\bar{X}) = \frac{1}{4}\sum_{i=1}^{4} E(X_i) = \frac{4 \cdot 10}{4} = 10.$$

Незалежність дає нульові коваріації, тому

$$\mathrm{Var}(\bar{X}) = \frac{1}{16}\sum_{i=1}^{4} \mathrm{Var}(X_i) = \frac{4 \cdot 50}{16} = 12.5.$$

Звідси стандартне відхилення середнього дорівнює $\sqrt{12.5} \approx 3.5355$. Емпірична дисперсія 1000 середніх становить 13.0804, що близько до 12.5. Незалежність потрібна саме для використаної формули дисперсії.

1(д). Нормальне наближення та центральна гранична теорема

Умова завдання

До гістограми середніх додайте теоретичну нормальну щільність із математичним сподіванням і дисперсією, знайденими в пункті 1(г). Порівняйте щільність із гістограмою. Поясніть, якого результату очікуєте і чому. Що зміниться, якщо збільшити обсяг кожної вибірки з 4 до 100? Обґрунтуйте відповідь за допомогою центральної граничної теореми.

Прогноз перед моделюванням.

Перед побудовою графіка очікую, що при $n = 4$ нормальна щільність лише приблизно описуватиме середні: асиметрія ще помітна. При $n = 100$ за ЦГТ наближення має бути кращим, а дисперсія середнього зменшиться з $50/4$ до $50/100$.

Код Python

```
x_grid = np.linspace(min(0, small_means.min()), small_means.max(), 500)
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.hist(small_means, bins=30, density=True, edgecolor="white", alpha=0.7,
        label="Середні Gamma(2, 5), n = 4")
ax.plot(x_grid, stats.norm.pdf(x_grid, loc=mean_of_mean, scale=np.sqrt(variance_of_mean)),
        color="darkred", linewidth=2, label="Нормальна щільність N(10, 12.5)")
ax.set(xlabel="Вибіркове середнє (умовні одиниці)", ylabel="Щільність (1 / ум. од.)",
       title="Нормальне наближення для середнього, n = 4")
ax.legend()
fig.tight_layout()
fig.savefig(FIGURES_DIR / "part1_normal_approximation.pdf")
plt.show()
```

Результати та інтерпретація

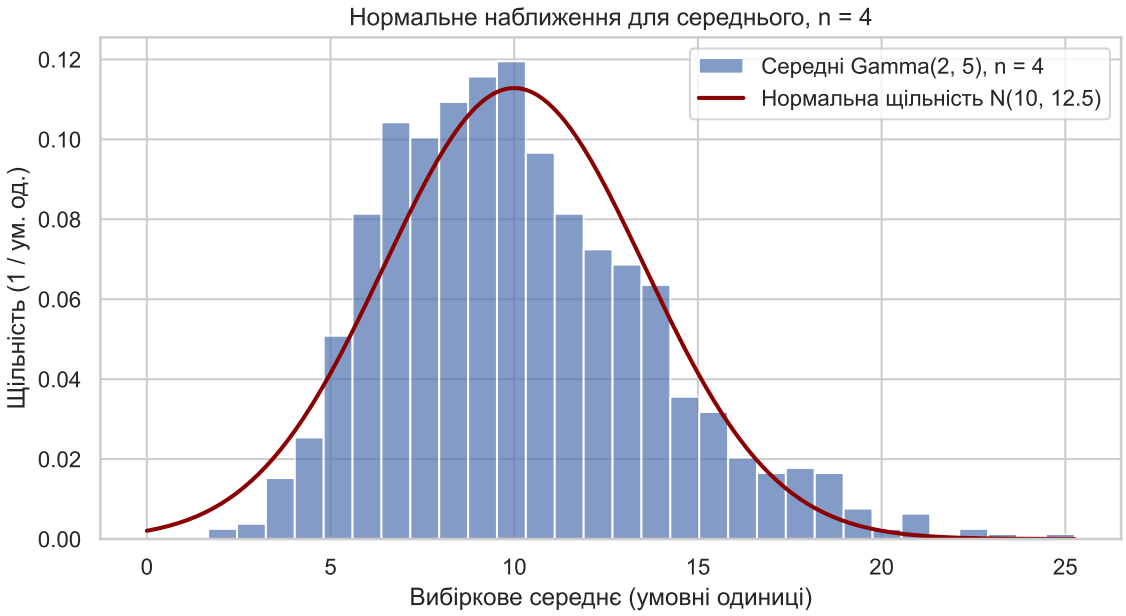


Рис. 4: Порівняння щільності середніх при обсязі 4 з нормальним наближенням.

На гістограму щільності накладено нормальну щільність $N(10, 12.5)$, де другий параметр позначає дисперсію. Її центр і масштаб близькі до емпіричних, однак гістограма при $n = 4$ залишається асиметричною; симетрична нормальна крива не відтворює правий хвіст і навіть допускає від'ємні середні. Це узгоджується з прогнозом перед побудовою графіка.

За ЦГТ для незалежних однаково розподілених величин зі скінченними сподіванням і дисперсією

$$\frac{\sqrt{n}(\bar{X}_n - 10)}{\sqrt{50}} \xrightarrow{d} N(0, 1).$$

При $n = 100$ очікуємо краще наближення $\bar{X}_{100} \approx N(10, 0.5)$: середнє залишається 10, а дисперсія дорівнює $50/100 = 0.5$. Саме збільшення обсягу кожної вибірки покращує нормальне наближення. Збільшення кількості повторень при незмінному $n = 4$ лише точніше показало б той самий ненормальний розподіл середнього.

Частина 2. Оцінювання максимальною правдоподібністю

Підготовка частини 2

Завантажуємо наданий CSV, перевіряємо потрібні колонки, бінарні значення та пропуски.

Код Python

```python
# Source: the app_metrics.csv supplied alongside assignment_2.pdf.
# Keep the source file unchanged; no external download is required.
DATA_PATH = INPUT_DIR / "app_metrics.csv"
if not DATA_PATH.is_file():
    raise FileNotFoundError(f"Place the supplied app_metrics.csv at {DATA_PATH}")
df = pd.read_csv(DATA_PATH)
required = {"Platform", "Session_Time", "Registration_Success", "Page_Views",
            "Purchase_Amount", "Revenue", "Subscription"}
missing = required.difference(df.columns)
if missing:
    raise ValueError(f"Missing required columns: {sorted(missing)}")
for column in ("Registration_Success", "Subscription"):
    if not df[column].dropna().isin([0, 1]).all():
        raise ValueError(f"{column} must contain binary values 0 or 1")
# Review missing values before choosing a per-analysis handling policy.
display(df.head())
display(df.isna().sum().rename("missing"))

from scipy.optimize import minimize
from statsmodels.stats.diagnostic import lilliefors

P_BOUNDS = [(1e-5, 1 - 1e-5)]
FUTURE_VISITORS = 20
```

2(a). Перевірка нормальності реєстрацій

Умова завдання

Завантажте колонку `Registration_Success` із `app_metrics.csv`. Виконайте одновибірковий критерій Колмогорова--Смирнова в модифікації Лілієфорса, коли параметри розподілу невідомі й оцінюються за вибіркою. Перевірте можливість нормального наближення цих бінарних даних. Запишіть H_0, H_1, статистику, p-значення та висновок.

Код Python

```python
registrations = df["Registration_Success"].dropna().to_numpy()
registration_d, registration_p = lilliefors(registrations, dist="norm", pvalmethod="table")
normality = pd.DataFrame({"n": [len(registrations)], "successes": [registrations.sum()],
                          "D": [registration_d], "p_returned": [registration_p],
                          "reject": [registration_p < ALPHA]})
display(normality)
normality.to_csv(RESULTS_DIR / "part2_normality.csv", index=False)
```

Результати та інтерпретація

У файлі 250 спостережень без пропусків; успішних реєстрацій 68. Припускаємо незалежність користувачів. За вимогою завдання перевіряємо H_0: розподіл спостережень нормальний з невідомими параметрами; H_1: він не є нормальним. Рівень значущості $\alpha = 0.05$.

Виклик `lilliefors(registrations, dist="norm", pvalmethod="table")` повернув $D = 0.457078$ та $p_{\text{returned}} = 0.001$. Це нижня межа табличної реалізації statsmodels: коректно записувати $p \le 0.001$, а не вважати 0.001 точним p-значенням. Формально тест відхиляє H_0.

Водночас бінарна величина має лише значення 0 і 1, тому не може мати неперервний нормальний розподіл. Тест Лілієфорса розрахований на неперервні дані, тому для бінарних значень 0 і 1 його p-значення може бути неточним. Для окремої реєстрації використовуємо розподіл Бернуллі. Нормальне наближення може стосуватися вибіркової частки при великому обсязі, а не окремих нулів та одиниць.

2(б). Правдоподібність Бернуллі та чисельна оцінка

Умова завдання

Запишіть функцію правдоподібності $L(p)$ і логарифмічну функцію $\ln L(p)$ для вибірки Бернуллі. Реалізуйте від'ємну логарифмічну правдоподібність

$$-\ln L(p) = -\sum_{t=1}^{T} \big(r_t \ln p + (1 - r_t) \ln(1 - p) \big).$$

Знайдіть оптимальну оцінку за допомогою `scipy.optimize.minimize` з обмеженням $p \in [10^{-5}, 1-10^{-5}]$. Порівняйте чисельну оцінку $\widehat{p}_{MLE}$ з вибірковим середнім. У вихідній умові після формули NLL вжито слово «максимізуйте»; напрям оптимізації пояснено у відповіді.

Код Python

```python
def negative_log_likelihood(p_array):
    p = p_array[0]
    return -np.sum(registrations * np.log(p) + (1 - registrations) * np.log1p(-p))

fit = minimize(negative_log_likelihood, x0=[0.5], bounds=P_BOUNDS, method="L-BFGS-B")
if not fit.success:
    raise RuntimeError(fit.message)
p_hat = float(fit.x[0])
sample_proportion = float(registrations.mean())
mle_results = pd.DataFrame({"p_hat": [p_hat], "sample_mean": [sample_proportion],
                            "absolute_difference": [abs(p_hat - sample_proportion)],
                            "negative_log_likelihood": [fit.fun]})
display(mle_results)
mle_results.to_csv(RESULTS_DIR / "part2_mle.csv", index=False)
```

Результати та інтерпретація

Нехай $r_t \in \{0, 1\}$, $T = 250$, $S = \sum_{t=1}^{T} r_t = 68$. Для незалежних випробувань зі спільною ймовірністю p:

$$L(p) = \prod_{t=1}^{T} p^{r_t} (1 - p)^{1 - r_t} = p^{68} (1 - p)^{182},$$

$$\ell(p) = \ln L(p) = 68 \ln p + 182 \ln(1 - p), \qquad \mathrm{NLL}(p) = -\ell(p).$$

Максимізація $\ell(p)$ еквівалентна **мінімізації** NLL; слово «максимізуйте» після від'ємного логарифма в умові є неточністю. Аналітично

$$\ell'(p) = \frac{68}{p} - \frac{182}{1 - p} = 0 \quad \Longrightarrow \quad \widehat{p} = \frac{68}{250} = 0.272.$$

Друга похідна $\ell''(p) = -68/p^2 - 182/(1 - p)^2 < 0$, тому це максимум. Чисельний мінімум `scipy.optimize.minimize` на $[10^{-5}, 1-10^{-5}]$ дає $\widehat{p}_{MLE} = 0.27199997$; вибіркове середнє дорівнює 0.272. Абсолютна різниця 3.2544×10^{-8} зумовлена чисельною похибкою оптимізації. Оцінена конверсія становить 27.2%.

2(в). Ймовірності кількості майбутніх реєстрацій

Умова завдання

За знайденою оцінкою $\widehat{p}$ обчисліть імовірності для наступних 20 відвідувачів:

- зареєструються рівно 5;
- зареєструються щонайменше 8;
- кількість реєстрацій буде строго більшою за $20\widehat{p}$.

Використайте PMF та CDF біноміального розподілу.

Код Python

```python
expected_registrations = FUTURE_VISITORS * p_hat
first_above_mean = int(np.floor(expected_registrations)) + 1
probabilities = pd.DataFrame({
    "event": ["K = 5", "K >= 8", f"K > 20*p_hat (K >= {first_above_mean})"],
    "probability": [stats.binom.pmf(5, FUTURE_VISITORS, p_hat),
                    stats.binom.sf(7, FUTURE_VISITORS, p_hat),
                    stats.binom.sf(first_above_mean - 1, FUTURE_VISITORS, p_hat)],
})
print(f"E(K) = {expected_registrations:.8f}; first integer above E(K): {first_above_mean}")
display(probabilities)
probabilities.to_csv(RESULTS_DIR / "part2_probabilities.csv", index=False)
```

Результати та інтерпретація

За припущення незалежності майбутніх відвідувачів і незмінної ймовірності реєстрації $K \sim \mathrm{Bin}(20, \widehat{p})$. Використовуємо знайдену чисельну оцінку $\widehat{p}$:

$$P(K = 5) = \binom{20}{5} \widehat{p}^{5} (1 - \widehat{p})^{15} = 0.197360,$$

$$P(K \geq 8) = 1 - F_K(7) = 0.150328.$$

Оскільки $E(K) = 20\widehat{p} \approx 5.44$, строго більше за сподівання означає щонайменше 6 реєстрацій:

$$P(K > 20\widehat{p}) = P(K \geq 6) = 1 - F_K(5) = 0.472429.$$

Ймовірності обчислено функціями `binom.pmf` та `binom.sf` (остання дорівнює $1 - \mathrm{CDF}$). Це модельний прогноз із підставленою оцінкою p, який не враховує невизначеність самої оцінки та можливі зміни поведінки користувачів.

Частина 3. Перевірка гіпотез про продуктові метрики

Підготовка частини 3

Завантажуємо наданий CSV, перевіряємо потрібні колонки, бінарні значення та пропуски.

Код Python

```python
# Source: the app_metrics.csv supplied alongside assignment_2.pdf.
# Keep the source file unchanged; no external download is required.
DATA_PATH = INPUT_DIR / "app_metrics.csv"
if not DATA_PATH.is_file():
    raise FileNotFoundError(f"Place the supplied app_metrics.csv at {DATA_PATH}")
df = pd.read_csv(DATA_PATH)
required = {"Platform", "Session_Time", "Registration_Success", "Page_Views",
            "Purchase_Amount", "Revenue", "Subscription"}
missing = required.difference(df.columns)
if missing:
    raise ValueError(f"Missing required columns: {sorted(missing)}")
for column in ("Registration_Success", "Subscription"):
    if not df[column].dropna().isin([0, 1]).all():
        raise ValueError(f"{column} must contain binary values 0 or 1")
# Review missing values before choosing a per-analysis handling policy.
display(df.head())
display(df.isna().sum().rename("missing"))

from statsmodels.stats.diagnostic import lilliefors
from statsmodels.stats.proportion import proportions_ztest
```

3(a). Чи перевищує середній час сесії 15 хвилин?

Умова завдання

Продакт-менеджер стверджує, що очікуваний час утримання користувача `Session_Time` перевищує 15 хвилин. Перевірте це твердження відповідним одностороннім параметричним критерієм при $\alpha = 0.05$. Наведіть гіпотези, формулу статистики з числовою підстановкою, критичну область, p-значення, рішення та бізнес-висновок.

Код Python

```python
session = df["Session_Time"].dropna()
n_session = len(session)
session_mean, session_sd = session.mean(), session.std(ddof=1)
t_session = (session_mean - 15) / (session_sd / np.sqrt(n_session))
session_df = n_session - 1
session_critical = stats.t.ppf(1 - ALPHA, session_df)
session_test = stats.ttest_1samp(session, popmean=15, alternative="greater")
assert np.isclose(t_session, session_test.statistic)
session_results = pd.DataFrame({"n": [n_session], "mean": [session_mean], "sd": [session_sd],
    "t": [t_session], "df": [session_df], "critical": [session_critical],
    "p": [session_test.pvalue], "reject": [session_test.pvalue < ALPHA]})
display(session_results)
session_results.to_csv(RESULTS_DIR / "part3_session.csv", index=False)
```

Результати та інтерпретація

Гіпотези: $H_0 : \mu \leq 15$ проти $H_1 : \mu > 15$; статистику обчислюємо на межі $\mu_0 = 15$. Дані містять $n = 250$ значень без пропусків. Припускаємо незалежну репрезентативну вибірку зі скінченною дисперсією. Точний t-критерій вимагає нормальності; при великому n використовуємо наближення для середнього.

Статистика та критична область:

$$t = \frac{\bar{x} - 15}{s/\sqrt{n}} = \frac{16.52012 - 15}{5.000818/\sqrt{250}} = 4.806255, \qquad df = 249.$$

Відхиляємо H_0, якщо $t > t_{0.95,249} = 1.650996$.

Результат: $p = 1.3306 \times 10^{-6} < 0.05$, тому H_0 відхиляємо. Є статистичні підстави вважати, що середній час сесії перевищує 15 хвилин. У вибірці середній час дорівнює 16.52012 хв, тобто приблизно на 1.52 хв більше за поріг. Висновок стосується середнього, а не кожного користувача, і спирається на припущення щодо вибірки.

3(б). Порівняння Revenue між iOS та Android

Умова завдання

Перевірте, чи відрізняється середній Revenue користувачів iOS від середнього Revenue користувачів Android. Генеральні дисперсії невідомі та не обов'язково рівні: використайте двосторонній t-критерій Уелша при $\alpha = 0.05$. Наведіть гіпотези, формулу статистики з числовою підстановкою, критичну область, p-значення, рішення та бізнес-висновок.

Код Python

```python
ios = df.loc[df["Platform"] == "iOS", "Revenue"].dropna()
android = df.loc[df["Platform"] == "Android", "Revenue"].dropna()
n_ios, n_android = len(ios), len(android)
mean_ios, mean_android = ios.mean(), android.mean()
var_ios, var_android = ios.var(ddof=1), android.var(ddof=1)
v_ios, v_android = var_ios / n_ios, var_android / n_android
welch_se = np.sqrt(v_ios + v_android)
t_welch = (mean_ios - mean_android) / welch_se
welch_df = (v_ios + v_android)**2 / (v_ios**2 / (n_ios - 1) + v_android**2 / (n_android - 1))
welch_critical = stats.t.ppf(1 - ALPHA / 2, welch_df)
welch_test = stats.ttest_ind(ios, android, equal_var=False, alternative="two-sided")
assert np.isclose(t_welch, welch_test.statistic)
welch_results = pd.DataFrame({"n_ios": [n_ios], "n_android": [n_android],
    "mean_ios": [mean_ios], "mean_android": [mean_android],
    "var_ios": [var_ios], "var_android": [var_android],
    "t": [t_welch], "df": [welch_df], "critical": [welch_critical],
    "p": [welch_test.pvalue], "reject": [welch_test.pvalue < ALPHA]})
display(welch_results.T)
welch_results.to_csv(RESULTS_DIR / "part3_welch.csv", index=False)
```

Результати та інтерпретація

Гіпотези: $H_0 : \mu_I = \mu_A$, $H_1 : \mu_I \neq \mu_A$, де I позначає iOS, A - Android. Використано 110 та 140 спостережень відповідно, пропусків немає. Припускаємо незалежність користувачів і груп та скінченні дисперсії; рівності дисперсій не вимагаємо. Для ненормальних сукупностей висновок є наближеним; обидві групи досить великі для застосування ЦГТ за цих умов.

Статистика Уелша:

$$t = \frac{\bar{x}_I - \bar{x}_A}{\sqrt{s_I^2/n_I + s_A^2/n_A}} = \frac{500.9221 - 393.5084}{\sqrt{65010.5061/110 + 17623.7712/140}} = 4.011751.$$

Число ступенів свободи оцінено за формулою

$$\nu = \frac{(s_I^2/n_I + s_A^2/n_A)^2}{(s_I^2/n_I)^2/(n_I - 1) + (s_A^2/n_A)^2/(n_A - 1)} = 154.869447.$$

Відхиляємо H_0, якщо $|t| > t_{0.975,\nu} = 1.975400$.

Результат: $p = 9.3545 \times 10^{-5} < 0.05$, тому H_0 відхиляємо. Дані свідчать про відмінність середнього Revenue між платформами. Середнє iOS вище приблизно на 107.41 грошових одиниць. Валюту в умові не вказано. Цей зв'язок не доводить, що саме платформа спричиняє різницю: групи можуть відрізнятися складом користувачів. Перевірка нормальності сукупного Revenue у пункті 3(д) не є окремою перевіркою нормальності кожної групи.

3(в). Лінійний зв’язок переглядів і суми покупки

Умова завдання

Перевірте гіпотезу про відсутність зв’язку між кількістю переглядів `Page_Views` і сумою покупки `Purchase_Amount` за допомогою критерію для коефіцієнта лінійної кореляції Пірсона при $\alpha = 0.05$. Наведіть гіпотези, формулу статистики з числовою підстановкою, критичну область, p-значення, рішення та бізнес-висновок.

Код Python

```python
paired = df[["Page_Views", "Purchase_Amount"]].dropna()
n_pairs = len(paired)
x, y = paired["Page_Views"], paired["Purchase_Amount"]
dx, dy = x - x.mean(), y - y.mean()
sum_xy, sum_xx, sum_yy = (dx * dy).sum(), (dx**2).sum(), (dy**2).sum()
r = sum_xy / np.sqrt(sum_xx * sum_yy)
t_correlation = r * np.sqrt((n_pairs - 2) / (1 - r**2))
correlation_critical = stats.t.ppf(1 - ALPHA / 2, n_pairs - 2)
correlation_test = stats.pearsonr(x, y)
assert np.isclose(r, correlation_test.statistic)
assert np.isclose(2 * stats.t.sf(abs(t_correlation), n_pairs - 2), correlation_test.pvalue)
correlation_results = pd.DataFrame({"n": [n_pairs], "sum_xy": [sum_xy],
    "sum_xx": [sum_xx], "sum_yy": [sum_yy], "r": [r], "t": [t_correlation],
    "df": [n_pairs - 2], "critical": [correlation_critical],
    "p": [correlation_test.pvalue], "reject": [correlation_test.pvalue < ALPHA]})
display(correlation_results.T)
correlation_results.to_csv(RESULTS_DIR / "part3_correlation.csv", index=False)
```

Результати та інтерпретація

Гіпотези: $H_0 : \rho = 0$, $H_1 : \rho \neq 0$. Перевіряємо саме лінійний зв'язок. Використано 250 пар Page_Views та Purchase_Amount; пропусків немає. Якби вони були, вилучалися б цілі пари. Припускаємо незалежність пар. Звичайна формула p-значення спирається на спільний нормальний розподіл двох ознак. Page_Views є кількістю переглядів, тому ця умова не виконується точно; p-значення використовуємо як наближення.

Статистика:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}} = \frac{88681.7496}{\sqrt{1940.9640 \cdot 4948498.8700}} = 0.904875,$$

$$t = r\sqrt{\frac{n-2}{1-r^2}} = 0.904875\sqrt{\frac{248}{1-0.904875^2}} = 33.476102.$$

За $df = 248$ критична область: $|t| > t_{0.975,248} = 1.969576$.

Результат: $p = 5.7422 \times 10^{-94} < 0.05$, тому за використаним критерієм H_0 відхиляємо. Спостерігаємо сильний додатний лінійний зв'язок: користувачі з більшою кількістю переглядів зазвичай мають вищу суму покупки. Це не доводить причинного впливу переглядів на покупку. Нульова кореляція також не означала б загальної незалежності.

3(г). Чи перевищує частка підписок 10%?

Умова завдання

Перевірте, чи генеральна частка користувачів із платною підпискою `Subscription` статистично значущо перевищує 10%. Використайте $\alpha = 0.05$. Наведіть гіпотези, формулу статистики з числовою підстановкою, критичну область, p-значення, рішення та бізнес-висновок.

Код Python

```python
subscription = df["Subscription"].dropna()
n_subscription = len(subscription)
count_subscription = int(subscription.sum())
p0 = 0.10
subscription_share = count_subscription / n_subscription
z_subscription = (subscription_share - p0) / np.sqrt(p0 * (1 - p0) / n_subscription)
z_builtin, p_subscription = proportions_ztest(count_subscription, n_subscription,
    value=p0, alternative="larger", prop_var=p0)
assert np.isclose(z_subscription, z_builtin)
subscription_critical = stats.norm.ppf(1 - ALPHA)
subscription_results = pd.DataFrame({"n": [n_subscription], "successes": [count_subscription],
    "share": [subscription_share], "expected_successes_null": [n_subscription * p0],
    "expected_failures_null": [n_subscription * (1 - p0)], "z": [z_subscription],
    "critical": [subscription_critical], "p": [p_subscription], "reject": [p_subscription < ALPHA]})
display(subscription_results.T)
subscription_results.to_csv(RESULTS_DIR / "part3_subscription.csv", index=False)
```

Результати та інтерпретація

Гіпотези: $H_0 : p \le 0.10$ проти $H_1 : p > 0.10$. Маємо 250 бінарних спостережень без пропусків та 34 підписки, тому $\widehat{p} = 34/250 = 0.136$. Припускаємо незалежні випробування Бернуллі зі спільною ймовірністю успіху. Для нормального наближення на межі H_0 перевіряємо $np_0 = 25$ та $n(1-p_0) = 225$; обидва значення перевищують 10.

Статистика:

$$z = \frac{\widehat{p} - p_0}{\sqrt{p_0(1-p_0)/n}} = \frac{0.136 - 0.10}{\sqrt{0.10 \cdot 0.90/250}} = 1.897367.$$

У `proportions_ztest` явно задано `prop_var=0.10`, щоб використати дисперсію за H_0. Критична область: $z > z_{0.95} = 1.644854$.

Результат: одностороннє $p = 0.028890 < 0.05$, тому H_0 відхиляємо. Дані підтримують твердження, що генеральна частка платних підписок перевищує 10%. Вибіркова частка дорівнює 13.6%, тобто на 3.6 відсоткового пункту вище за поріг. Це наближений висновок за припущень тесту; нормально розподілені окремі бінарні спостереження не потрібні.

3(д). Перевірка нормальності Revenue

Умова завдання

Перевірте нормальність метрики `Revenue` одновибірковим критерієм Колмогорова-Смирнова в модифікації Лілієфорса. Використайте виключно вбудовану функцію `statsmodels`, без ручного обчислення статистики, при $\alpha = 0.05$. Наведіть гіпотези, отриману статистику, p-значення та висновок щодо припущень попередніх тестів.

Код Python

```python
revenue = df["Revenue"].dropna()
revenue_d, revenue_p = lilliefors(revenue, dist="norm", pvalmethod="table")
revenue_results = pd.DataFrame({"n": [len(revenue)], "D": [revenue_d],
    "p_returned": [revenue_p], "reject": [revenue_p < ALPHA]})
display(revenue_results)
revenue_results.to_csv(RESULTS_DIR / "part3_revenue_normality.csv", index=False)
```

Результати та інтерпретація

Гіпотези: H_0: Revenue має нормальний розподіл з невідомими параметрами; H_1: розподіл відрізняється від нормального. Використано 250 спостережень без пропусків; припускаємо незалежність і однаковий розподіл спостережень.

Вбудована функція: `lilliefors` із модуля `statsmodels.stats.diagnostic`.

Виклик: `lilliefors(revenue, dist="norm", pvalmethod="table")`.

Статистику вручну не обчислюємо, як вимагає цей підпункт. Отримано $D = 0.146746$ та $p_{\text{returned}} = 0.001$. Це нижня таблична межа, тому записуємо $p \leq 0.001$. Правило відхилення: $p < 0.05$.

Результат: нормальність сукупного Revenue відхиляємо. Просту нормальну модель розподілу індивідуального доходу ці дані не підтримують. Проте це не автоматично скасовує всі попередні висновки. Критерії для середніх можуть спиратися на ЦГТ при великих незалежних вибірках зі скінченною дисперсією; для Уелша важливі розподіли всередині платформ. Критерій Пірсона має окремі вимоги до спільного розподілу двох ознак. Критерій для частки використовує бінарні дані та нормальне наближення частки, а не нормальність окремих значень. Твердження умови про обов'язкову нормальність усіх ознак потребує цього уточнення.

Усі перевірки частини 3 виконано окремо при $\alpha = 0.05$, як задано. Спільний рівень помилки для всієї серії тестів не контролюється.

Частина 4. Емпіричний рівень помилки першого роду

Підготовка частини 4

Після спільного блоку налаштування задаємо параметри експерименту.

Код Python

```python
NULL_MEAN = 100
TRUE_VARIANCE = 25
TRUE_SD = np.sqrt(TRUE_VARIANCE)
N = 30
REPLICATIONS = 5000
```

4(a). Двосторонній t-критерій для однієї вибірки

Умова завдання

Змоделюйте вибірку обсягу $n = 30$ із нормального розподілу з $\mu_0 = 100$ і $\sigma^2 = 25$. Перевірте $H_0 : \mu = 100$ проти двосторонньої альтернативи, вважаючи генеральну дисперсію невідомою. Вручну обчисліть тестову статистику, визначте аналітичну критичну область для $\alpha = 0.05$ і прийміть рішення.

Код Python

```python
sample = rng.normal(loc=NULL_MEAN, scale=TRUE_SD, size=N)
sample_mean, sample_sd = sample.mean(), sample.std(ddof=1)
t_statistic = (sample_mean - NULL_MEAN) / (sample_sd / np.sqrt(N))
critical = stats.t.ppf(1 - ALPHA / 2, df=N - 1)
p_value = 2 * stats.t.sf(abs(t_statistic), df=N - 1)
single_result = pd.DataFrame({"n": [N], "mean": [sample_mean], "sd": [sample_sd],
    "t": [t_statistic], "df": [N - 1], "critical": [critical], "p": [p_value],
    "reject": [abs(t_statistic) > critical]})
display(single_result)
single_result.to_csv(RESULTS_DIR / "part4_single.csv", index=False)
```

Результати та інтерпретація

Моделюємо $X_i \sim N(100, 25)$, тобто генератор використовує стандартне відхилення 5. У тесті дисперсію вважаємо невідомою. Гіпотези: $H_0 : \mu = 100$, $H_1 : \mu \neq 100$.

$$t = \frac{\bar{x} - 100}{s/\sqrt{30}} = \frac{100.084071 - 100}{3.883745/\sqrt{30}} = 0.118564.$$

За незалежності та нормальності при H_0 маємо $T \sim t_{29}$. Критична область рівня 0.05:

$$(-\infty, -t_{0.975,29}) \cup (t_{0.975,29}, \infty), \qquad |t| > 2.045230.$$

Отримано $p = 0.906438$, тому H_0 не відхиляємо. У цій вибірці не виявлено статистично значущої відмінності середнього від 100. Невідхилення не є доведенням рівності.

4(б). 5000 повторень під нульовою гіпотезою

Умова завдання

Незалежно змоделюйте $M = 5000$ вибірок обсягу $n = 30$ із $\mu_0 = 100$ і $\sigma^2 = 25$. Для кожної виконайте тест і збережіть індикатор p_i: 1 при відхиленні H_0, 0 в іншому випадку. Обчисліть $\widehat{\alpha} = M^{-1} \sum_{i=1}^{M} p_i$ та порівняйте з $\alpha = 0.05$. Поясніть, чи очікуєте велику різницю, з огляду на відповідність умов експерименту припущенням критерію Стьюдента.

Прогноз перед моделюванням.

Перед моделюванням очікую невелику випадкову різницю між емпіричним рівнем та 0.05: дані незалежні й нормальні, тому умови точного t-критерію виконано. Кількість повторень скінченна, тому точного збігу не очікую.

Код Python

```
null_samples = rng.normal(loc=NULL_MEAN, scale=TRUE_SD, size=(REPLICATIONS, N))
t_statistics = (null_samples.mean(axis=1) - NULL_MEAN) / (null_samples.std(axis=1, ddof=1) / np.sqrt(N))
rejection_indicators = (np.abs(t_statistics) > critical).astype(int)
empirical_alpha = rejection_indicators.mean()
mc_se = np.sqrt(ALPHA * (1 - ALPHA) / REPLICATIONS)
size_results = pd.DataFrame({"M": [REPLICATIONS], "rejections": [rejection_indicators.sum()],
    "empirical_alpha": [empirical_alpha], "nominal_alpha": [ALPHA],
    "mc_se_under_null": [mc_se], "difference_in_se": [(empirical_alpha - ALPHA) / mc_se]})
display(size_results.T)
size_results.to_csv(RESULTS_DIR / "part4_size.csv", index=False)
pd.DataFrame({"t": t_statistics, "reject": rejection_indicators}).to_csv(
    RESULTS_DIR / "part4_replications.csv", index=False)
```

Результати та інтерпретація

Для кожної з $M = 5000$ незалежних вибірок обсягу 30 обчислено t-статистику та індикатор $I_i = \mathbf{1}\{|T_i| > t_{0.975,29}\}$. Ці індикатори відповідають позначенню p_i в умові; вони не є p-значеннями.

$$\widehat{\alpha} = \frac{1}{M} \sum_{i=1}^{M} I_i = \frac{266}{5000} = 0.0532.$$

Порівняно з 0.05 різниця становить 0.0032, тобто 0.32 відсоткового пункту. Для правильно відкаліброваного критерію стандартна похибка моделювання дорівнює

$$\mathrm{SE}_{MC} = \sqrt{\frac{0.05(1 - 0.05)}{5000}} \approx 0.003082.$$

Спостережена різниця дорівнює приблизно 1.04 такої похибки, тому вона узгоджується зі звичайними випадковими коливаннями. Прогноз перед симуляцією підтвердився: нормальність, незалежність і оцінювання дисперсії з дільником $n-1$ забезпечують точний t-критерій, а скінченна кількість повторень дає невелике відхилення емпіричної частки від 0.05.

Частина 5. Потужність критерію Лілієфорса

Підготовка частини 5

Після спільного блоку налаштування задаємо параметри експерименту.

Код Python

```python
from statsmodels.stats.diagnostic import lilliefors

N = 80
REPLICATIONS = 1000
DEGREES_OF_FREEDOM = np.arange(2, 61)
# ALPHA = 0.05 follows the preceding testing sections; record this convention.
# Full assignment: 59 × 1000 = 59,000 normality tests.
```

Умова завдання

Для кожного $df = 2, 3, \ldots, 60$ змоделюйте $M = 1000$ вибірок обсягу $n = 80$ з t-розподілу Стьюдента. Для кожної вибірки запустіть критерій Лілієфорса на нормальність і підрахуйте кількість правильних відхилень H_0. Поясніть, як ця кількість дозволяє оцінити потужність $1 - \beta$. Побудуйте графік потужності залежно від df та поясніть надійність і чутливість критерію при великих df, коли t-розподіл наближається до нормального.

Прогноз перед моделюванням.

Перед моделюванням очікую високу потужність при малих df через важкі хвости t-розподілу та її зниження зі зростанням df. Крива може коливатися через випадковість симуляції. Використовую рівень значущості $\alpha = 0.05$, як у попередніх частинах.

Код Python

```python
power_rows = []
for degrees in DEGREES_OF_FREEDOM:
    samples = rng.standard_t(df=degrees, size=(REPLICATIONS, N))
    rejections = 0
    for sample in samples:
        _, p_value = lilliefors(sample, dist="norm", pvalmethod="table")
        rejections += int(p_value < ALPHA)
    power_rows.append({"df": degrees, "rejections": rejections,
                       "power": rejections / REPLICATIONS})
power_results = pd.DataFrame(power_rows)
power_results["mc_se"] = np.sqrt(power_results["power"] * (1 - power_results["power"]) / REPLICATIONS)
power_results["beta_estimate"] = 1 - power_results["power"]
display(power_results)
power_results.to_csv(RESULTS_DIR / "part5_power.csv", index=False)
print(f"Виконано тестів: {len(DEGREES_OF_FREEDOM) * REPLICATIONS}")

fig, ax = plt.subplots(figsize=(9, 4.8))
ax.plot(power_results["df"], power_results["power"], "o-", markersize=3,
        label="Оцінена потужність")
ax.axhline(ALPHA, color="black", linestyle="--", label="Рівень значущості α = 0.05")
ax.set(xlabel="Число ступенів свободи df", ylabel="Частка відхилень нормальності",
       title="Потужність критерію Лілієфорса: n = 80, M = 1000", ylim=(0, 1))
ax.legend()
fig.tight_layout()
fig.savefig(FIGURES_DIR / "part5_power.pdf")
plt.show()
```

Результати та інтерпретація

Для кожного $df = 2, 3, \ldots, 60$ згенеровано 1000 незалежних вибірок обсягу 80. Для кожної викликано тест Лілієфорса з оціненими за вибіркою параметрами нормального розподілу. Гіпотези: H_0 - нормальність; H_1 - ненормальний розподіл. За будь-якого скінченного df t-розподіл відрізняється від нормального, тому відхилення H_0 є правильним рішенням.

$$\widehat{1-\beta}(df) = \frac{\#\{p\text{-значень} < 0.05\}}{1000}, \qquad \widehat{\beta}(df) = 1 - \widehat{1-\beta}(df).$$

Отже, частка відхилень оцінює **потужність**, а не саму помилку другого роду. Загалом виконано $59 \cdot 1000 = 59000$ тестів; усі 59 результатів збережено в таблиці.

При $df = 2$ отримано 897 відхилень і потужність 0.897; при $df = 5$ - 0.257; при $df = 10$ - 0.078; при $df = 60$ - 0.037, отже $\widehat{\beta} = 0.963$. Загальне зниження узгоджується з прогнозом: при малих df важкі хвости добре відрізняються від нормальних, а зі зростанням df t-розподіл наближається до нормального. При $df = 2$ генеральна дисперсія нескінченна, але тест усе одно обчислюється за скінченною вибіркою.

Оцінки потужності можуть коливатися через випадковість моделювання, тому не мають спадати строго монотонно. Коли t-розподіл наближається до нормального, частка відхилень наближається до частки помилкових відхилень для нормальних даних. Вона не обов'язково точно дорівнює заданим 5%.

Додаткова перевірка на нормальних вибірках обсягу $n = 80$ показала, що використана реалізація критерію відхиляє істинну H_0 приблизно у 3.5% випадків за заданого рівня 5%. Це разом із випадковими коливаннями допомагає пояснити потужність 0.037 при $df = 60$. Код і результати контрольної перевірки підготовлено для індивідуального захисту.

Отже, при $n = 80$ критерій добре помічає сильні відхилення, але слабко розпізнає близькі до нормального t-розподіли. Невідхилення нормальності для великих df не доводить її істинності та не гарантує відсутності важчих хвостів.

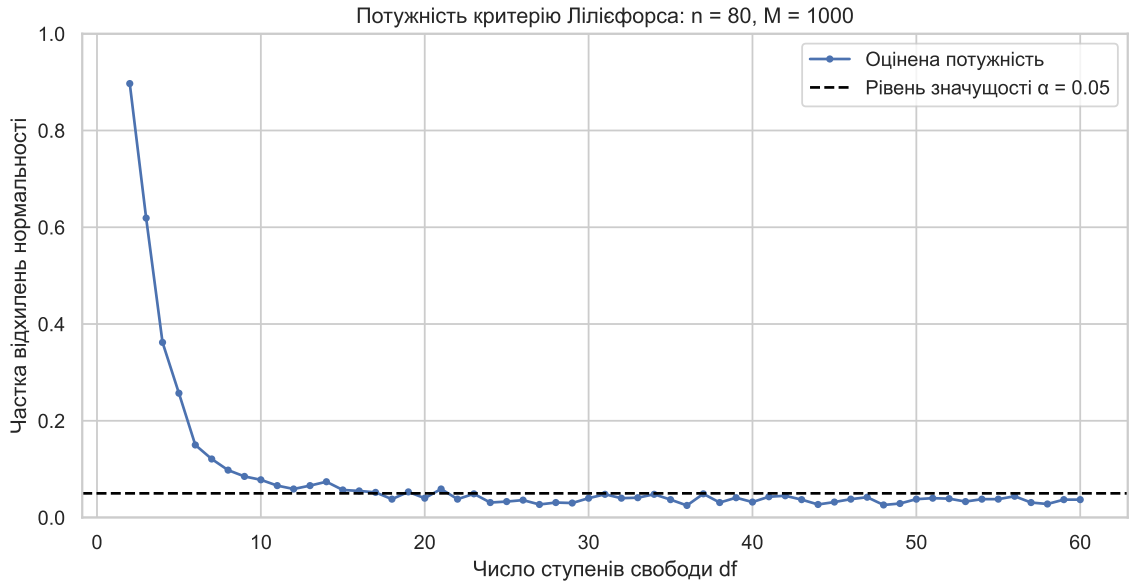


Рис. 5: Емпірична потужність проти t-альтернатив: усі ступені свободи від 2 до 60.

Повні результати моделювання потужності

Для кожного значення df виконано 1000 тестів. R - кількість відхилень нормальності, $\widehat{\pi} = R/1000$ - оцінка потужності.

Табл. 3: Результати всіх 59 серій симуляцій.

df	R	$\widehat{\pi}$	df	R	$\widehat{\pi}$	df	R	$\widehat{\pi}$
2	897	0.897	22	38	0.038	42	45	0.045
3	619	0.619	23	49	0.049	43	37	0.037
4	362	0.362	24	31	0.031	44	27	0.027
5	257	0.257	25	33	0.033	45	32	0.032
6	150	0.150	26	36	0.036	46	38	0.038
7	121	0.121	27	27	0.027	47	42	0.042
8	98	0.098	28	31	0.031	48	26	0.026
9	85	0.085	29	30	0.030	49	29	0.029
10	78	0.078	30	40	0.040	50	38	0.038
11	66	0.066	31	48	0.048	51	40	0.040
12	59	0.059	32	40	0.040	52	39	0.039
13	66	0.066	33	41	0.041	53	33	0.033
14	74	0.074	34	48	0.048	54	38	0.038
15	57	0.057	35	37	0.037	55	38	0.038
16	55	0.055	36	25	0.025	56	44	0.044
17	52	0.052	37	49	0.049	57	31	0.031
18	38	0.038	38	31	0.031	58	28	0.028
19	53	0.053	39	41	0.041	59	37	0.037
20	40	0.040	40	32	0.032	60	37	0.037
21	59	0.059	41	43	0.043			